

Introduction to APIs

Comprehensive Guide

 **How to Use These Notes:** Read each topic carefully, study the technical examples, and test yourself using the Q&A section at the bottom. Highlighted answers in green indicate the correct option.

Table of Contents

- 1. Introduction to APIs
- 2. API Types & Real-World Uses
- 3. How APIs Work (Requests & Responses)

1. Introduction to APIs

a. What is an API?

Topic: Definition and Concept of API.

Detailed Description: API stands for **Application Programming Interface**. It is a set of rules and protocols that allows different software applications to communicate with each other. It acts as an intermediary layer that processes data transfers between systems, abstracting the underlying complexity.



The Restaurant Analogy:

Think of an API as a **waiter** in a restaurant.

- **You (The Client):** You are at the table requiring food (data). You cannot go into the kitchen to cook it yourself.
- **The Kitchen (The Server):** This is where the food is prepared and ingredients are stored (database).
- **The Waiter (The API):** The waiter takes your order (request), delivers it to the kitchen, and brings the food back to you (response).

Without the waiter (API), the customer (Client) has no standard way to communicate with the kitchen (Server).

b. Why Do We Need APIs?

APIs are fundamental to modern software engineering for several reasons:

✅ Advantages:

- **Abstraction:** Hides complex internal logic. A developer using the Stripe API doesn't need to know how banking networks work.
- **Scalability:** Modular systems are easier to scale. Frontend and Backend can grow independently.
- **Automation:** Enables machines to talk to machines without human intervention.

⚠️ Challenges:

- **Security Risks:** APIs are gateways to data; if poorly secured, they become attack vectors.
- **Dependency:** If a third-party API changes or goes down, your application breaks.
- **Latency:** Network calls take more time than local function calls.

c. Where APIs Are Used

APIs operate behind the scenes of almost every digital interaction.

 Domain	 Use Case Example	 API Used
Social Media	Displaying your latest Tweets on a personal website	Twitter (X) API
Payments	Processing a credit card payment on an e-commerce site	Stripe / PayPal API
Maps	Showing the route of an Uber driver	Google Maps API
Weather	Phone widget showing 7-day forecast	OpenWeatherMap API
Authentication	"Log in with Google" button	OAuth 2.0 API

🔧 Available Tools & Libraries:

- **Postman:** The most popular tool for testing and exploring APIs.
- **cURL:** Command-line tool for making API requests.
- **Swagger/OpenAPI:** For documenting API specifications.

Reference Resources:

- [MDN Web Docs: Intro to APIs](#)
- [Postman: What is an API?](#)

2. API Types & Real-World Uses

a. Open APIs (Public APIs)

Topic: APIs available to the general public with minimal restrictions.

Detailed Description: Also known as External APIs, Open APIs are interfaces that any developer can freely access (sometimes requiring a free API key). They encourage innovation by allowing the developer community to build on top of existing services. Major tech companies like Google, Twitter, NASA publish Open APIs.

✦ **Example 1 – NASA Open API:** A student project fetches the Astronomy Picture of the Day by calling https://api.nasa.gov/planetary/apod?api_key=DEMO_KEY. The API returns the image URL and description in JSON format.

✦ **Example 2 – OpenWeatherMap API:** A weather dashboard app calls api.openweathermap.org/data/2.5/weather?q=London to display real-time temperature, wind, and humidity.

✓ **Advantages:**

- Free or low-cost — great for startups and students.
- Well-documented, easy to get started quickly.
- Large developer communities and support forums.
- Accelerates innovation (thousands of apps built on Maps, Twitter APIs).

⚠ **Challenges / Disadvantages:**

- Rate limiting — your app may be allowed for a certain number of calls per day.
- Service may be discontinued (e.g., Twitter API went paid in 2023).
- Privacy concerns when routing user data through third-party services.
- No SLA (Service Level Agreement) — no uptime guarantee.

b. Partner APIs

Topic: Restricted APIs shared only with approved business partners.

Detailed Description: Partner APIs are not publicly listed. They require a formal business agreement and specific authentication credentials. They are designed for B2B (Business-to-Business) integration and usually come with higher rate limits, dedicated support, and SLAs.

✦ **Example 1 – PayPal Partner API:** A large e-commerce platform (like Shopify) signs an agreement with PayPal and uses the PayPal Partner API to offer PayPal as a checkout option to merchants.

✦ **Example 2 – Salesforce Partner API:** A CRM software company integrates with Salesforce's Partner API to allow their users to sync customer records directly into Salesforce.

✓ **Advantages:**

- More secure — only vetted partners can access.
- Dedicated support and guaranteed uptime (SLA).
- Higher rate limits than public APIs.
- Enables powerful B2B revenue streams.

⚠ **Challenges / Disadvantages:**

- Requires slow formal onboarding (business agreements).
- Not available to individual developers or startups without partnerships.
- More costly to integrate due to complex requirements.

c. Internal APIs (Private APIs)

Topic: APIs that connect systems exclusively within a single organization.

Detailed Description: Internal APIs (also called Private APIs) are hidden from the public internet. They connect different internal departments, microservices, or data systems within a company. They are the foundation of modern microservices architecture where large monolithic applications are split into smaller, API-connected services.

✦ **Example 1 – Netflix Internal API:** Netflix's recommendation engine (a microservice) uses an internal API to fetch viewing history from the user profile service and compute personalized content recommendations.

✦ **Example 2 – HR & Payroll Integration:** A company's attendance management system calls an internal API to send daily attendance data to the payroll service, which then auto-calculates and releases salaries.

✓ Advantages:

- Maximum security — never exposed to the internet.
-
- Full control over design, versioning, and usage.
- Enables microservices architecture — each service can be developed independently.
- Faster development as teams can work on separate services in parallel.

⚠ Challenges / Disadvantages:

- High maintenance overhead — all teams must keep them updated and compatible.
- Documentation often neglected since it's "internal only."
- Service dependency chains — if one internal API fails, multiple systems break.

d. Composite APIs

Topic: APIs that bundle multiple service calls into one single request.

Detailed Description: In microservices-based systems, a single user screen may require data from 3–5 different services. Making separate API calls for each one increases latency and server load. Composite APIs solve this by merging multiple calls into one, reducing round-trips and improving performance — critical for mobile apps on slow connections.

✦ **Example 1 – E-Commerce Product Page:** Loading a product page requires: product details + stock availability + customer reviews + related products. A Composite API at `/product-page/{id}` returns all four in a single JSON response.

✦ **Example 2 – Travel Booking App:** A single button click in a travel app triggers a Composite API that simultaneously fetches available flights, hotel options, and car rentals from three different microservices.

✓ Advantages:

- Fewer network round-trips → faster page load times.
- Reduces battery/data usage on mobile apps.
- Simplifies the client-side codebase.
- Better user experience with faster data loading.

⚠ Challenges / Disadvantages:

- Server-side complexity increases — more logic to maintain.

- Debugging is harder when one of the bundled services fails silently.
- If one sub-service is slow, it blocks the entire composite response.

Tools for Managing API Types:

- **RapidAPI:** A marketplace to find Open and Partner APIs.
- **Kong / Apigee:** Enterprise API Gateways to manage Internal and Partner APIs.

Reference Resources:

- [RapidAPI: 4 Types of APIs](#)
- [IBM: API Guide](#)

3. How APIs Work (Requests & Responses)

a. API Architectural Styles

Topic: Different design patterns used to build APIs.

Detailed Description: An API architectural style defines the rules for how clients and servers communicate — what format data should be in, what protocols to use, and how requests/responses are structured. Choosing the right architecture depends on your application's needs (speed, security, flexibility).

Style	Full Name	Key Characteristic	Best Used For
 REST	Representational State Transfer	Stateless, uses HTTP methods, returns JSON	Web apps, Mobile APIs
 SOAP	Simple Object Access Protocol	Strict XML-based protocol with WS-Security	Banking, Healthcare, Enterprise
 GraphQL	Graph Query Language	Client specifies exact fields to fetch	Social networks, Mobile, Complex data
 gRPC	Google Remote Procedure Call	Binary protocol (Protobuf), very fast	Microservices, Low-latency systems

Style	Full Name	Key Characteristic	Best Used For
 WebSocket	WebSocket Protocol	Persistent bidirectional connection	Chat apps, Live sports scores, Gaming

✓ Advantages of Defined Architecture:

- Consistency across the team — all developers follow the same patterns.
- Easier to document, test, and maintain.
- Right architecture choice = better performance (e.g., gRPC for microservices).

⚠ Challenges:

- Wrong architecture choice leads to performance issues later.
- Migrating from SOAP to REST requires complete API redesign.
- Teams need training to use newer styles like GraphQL or gRPC.

🔧 Available Tools & Libraries for Architectural Styles:

- **REST:** Express.js (Node.js), Django REST Framework (Python), Spring Boot (Java), FastAPI (Python)
- **SOAP:** Apache CXF (Java), Zeep (Python), SoapUI (Testing)
- **GraphQL:** Apollo Server, Hasura, Strawberry (Python), GraphQL.js
- **gRPC:** gRPC-Go, grpc (Python), gRPC-Java
- **WebSocket:** Socket.io (Node.js), ws (Node.js), websockets (Python)

📚 Reference Resources:

- [RESTful API Design Guide – restfulapi.net](https://restfulapi.net)
- [GraphQL Official Documentation](https://graphql.org/learn/)
- [gRPC Official Documentation](https://grpc.io/docs/guides/)
- [MDN WebSockets API Guide](https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API)

b. Client–Server Model

Topic: The fundamental architecture behind all API communication.

Detailed Description: The Client-Server model separates the application into two roles: the **Client** (requester) and the **Server** (provider). Clients never directly access a database —

they go through the API layer. This separation enables independent scaling, security, and maintenance of each side.

CLIENT

(Browser / Mobile App)

1. HTTP Request
2. HTTP Response (JSON)

SERVER

(API + Database)

✦ **Example 1 – Weather Mobile App:** Your phone app (Client) sends GET request to `api.openweathermap.org/weather?city=Paris`. The Server looks up Paris data and returns temperature, humidity, conditions in JSON.

✦ **Example 2 – Online Banking:** Your browser (Client) sends POST to the bank's API to transfer money. The Server validates your JWT token, processes the transaction, updates the database, and returns success/failure.

✅ **Advantages:**

- Clear separation of concerns — frontend and backend teams work independently.
- Multiple clients (web, iOS, Android, IoT) share the same server API.
- Stateless design makes the server easier to scale horizontally (add more servers).

⚠️ **Challenges:**

- Network latency adds delays compared to in-process function calls.
- Authentication must be handled carefully at every request.
- A server outage impacts all clients simultaneously.

c. API Request Components

Topic: The individual components that make up a complete API request.

Detailed Description: Every API call you make is actually a structured HTTP message composed of multiple parts. Understanding each part is essential for any backend or frontend developer to correctly build, test, and debug APIs.

GETRetrieve**POST**Create**PUT**Replace**PATCH**Update**DELETE**Remove

1. **Endpoint (URL):** The address (e.g., `https://api.github.com/users`).
2. **Method:** The action to perform (GET, POST, etc.).

3. **Headers:** Metadata (e.g., Authorization: Bearer token123, Content-Type: application/json).
4. **Body/Payload:** The data sent to the server (used in POST/PUT).

Common HTTP Status Codes:

-  **200 OK:** Success.
-  **201 Created:** Resource successfully created.
-  **400 Bad Request:** Client sent invalid data.
-  **401 Unauthorized:** Missing or invalid authentication.
-  **404 Not Found:** Resource does not exist.
-  **500 Internal Server Error:** Server crashed or failed.

Example 1: Standard REST GET Request

GET /api/users/123 HTTP/1.1

Host: api.myservice.com

Authorization: Bearer abcd12345

Accept: application/json

Example 2: JSON Response from Server

HTTP/1.1 200 OK

Content-Type: application/json

```
{  
  "status": "success",  
  "data": {  
    "id": 123,  
    "name": "Jane Doe",  
    "role": "Engineer"  
  }  
}
```

Example 3: POST Request (Create New User)

POST /api/users HTTP/1.1

Host: api.myservice.com

Authorization: Bearer abcd12345

Content-Type: application/json

```
{  
  "name": "Alice Smith",  
  "email": "alice@example.com",  
  "role": "Student"  
}
```

--- Response ---

HTTP/1.1 201 Created

```
{  
  "status": "created",  
  "id": 456  
}
```

d. REST vs SOAP vs GraphQL

Comparison of the three dominant API styles:

Feature	 REST	 SOAP	 GraphQL
Full Name	Representational State Transfer	Simple Object Access Protocol	Graph Query Language
Type	Architectural Style	Protocol	Query Language
Data Format	JSON (most common), XML, HTML	XML Only	JSON

Feature	 REST	 SOAP	 GraphQL
Transport	HTTP / HTTPS	HTTP, SMTP, TCP	HTTP / HTTPS
Complexity	Low (Easy to learn)	High (Verbose, Strict)	Medium (Learning curve)
Performance	High (Cacheable)	Lower (Heavy XML parsing)	High (No over-fetching)
Best For	Public Web Services, Mobile	Enterprise, Banking, Security	Complex Data, Mobile, Social
Real Example	Twitter API, Google Maps	Salesforce (Legacy), Banking	GitHub GraphQL API, Facebook

Example: GraphQL Query (Fetch exact fields only)

```

query {
  user(id: "123") {
    name
    email
    friends {
      name
    }
  }
}

```

--- Response (Only what was requested!) ---

```

{
  "data": {
    "user": {

```

```
"name": "Jane Doe",  
"email": "jane@example.com",  
"friends": [  
  { "name": "Alice" },  
  { "name": "Bob" }  
]  
}  
}  
}
```

✦ Example: SOAP Request (XML Envelope)

```
<?xml version="1.0" encoding="UTF-8"?>  
  
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">  
  
  <soap:Body>  
  
    <GetUserRequest>  
  
      <userId>123</userId>  
  
    </GetUserRequest>  
  
  </soap:Body>  
  
</soap:Envelope>
```

✅ REST Advantages:

- Simple to learn — uses standard HTTP everyone knows.
- Stateless — easier to scale horizontally.
- Responses are cacheable (improves performance).
- Supported by every programming language and framework.

⚠️ REST Disadvantages:

- Over-fetching: Server may return more data than needed.
- No built-in real-time support (need WebSockets for live data).
- Multiple endpoints needed for different resources.

✅ SOAP Advantages:

- Built-in WS-Security standard — ideal for financial transactions.
- ACID-compliant — reliable for critical operations (bank transfers).
- Formal WSDL contract ensures both sides agree on data format.
- Works over multiple protocols (HTTP, SMTP, TCP).

⚠ **SOAP Disadvantages:**

- Verbose XML makes messages large and slow to parse.
- High learning curve — requires extensive tooling.
- Not suitable for lightweight mobile or web applications.
- Tightly coupled — any change to WSDL breaks existing clients.

✅ **GraphQL Advantages:**

- No over-fetching or under-fetching — client gets exactly what it asks for.
- Single endpoint for all operations (one URL for everything).
- Strongly typed schema serves as built-in documentation.
- Evolve API without versioning — just add new fields.

⚠ **GraphQL Disadvantages:**

- More complex server setup compared to REST.
- HTTP caching is harder since all requests go to one endpoint.
- N+1 query problem — can cause performance issues without DataLoader.
- Overkill for simple CRUD APIs.

🔧 **Tools & Libraries:**

- **REST:** Express.js (Node), Flask (Python).
- **SOAP:** Apache CXF, SoapUI.
- **GraphQL:** Apollo Server, Hasura.

📖 **Reference Resources:**

- [RESTfulAPI.net Guide](#)
- [GraphQL Official Docs](#)